

南开大学

本科生课程论文

中文题目: 知序:基于 HarmonyOS 的本地优先学习管理应用
面向大学生的资料·任务·专注·复盘学习闭环

外文题目: Design and Implementation of ZhiXu
A Local-First Study-Management App on HarmonyOS

学号: 2413575
姓名: 柯云超
年级: 2024 级
专业: 计算机科学与技术
系别: 计算机科学与技术
学院: 计算机学院
指导教师: 刘嘉欣, 董前琨, 张彪
完成日期: 2026 年 5 月

摘 要

随着移动互联网、智能终端和生成式人工智能的普及，大学生的学习活动越来越依赖移动设备完成。课程资料、实验任务、网页链接、课堂笔记和复习计划通常分散在多个工具中，导致学生在学习前需要反复寻找资料，在学习中难以维持专注，在学习后缺少稳定的复盘依据。现有待办、笔记和番茄钟工具多聚焦单点能力，难以把资料整理、任务分解、专注执行和学习复盘组织为连续流程。因此，如何在弱网甚至离线条件下构建一个稳定、可解释、可扩展的移动学习闭环，是本文关注的主要问题。

针对上述问题，本文设计并实现了一款名为“知序”的 HarmonyOS 学习管理应用。系统采用本地优先架构，以端侧关系型数据库 RDB 作为核心数据源，以 Preferences 保存运行时配置，并按照 MVVM 模式组织模型、视图和控制逻辑。应用提供今日看板、任务管理、资料库、番茄专注、复盘统计和云同步等功能：学习资料可被保存到本地并在远程 AI 可用时提炼为摘要和要点，资料提炼结果可进一步生成任务，任务完成过程通过番茄记录写入本地数据库，并最终反映到复盘统计中。后端采用 Spring Boot、MyBatis 与 MySQL/H2 数据库实现用户、项目、任务、番茄记录和同步接口，为跨设备备份和全栈展示提供支撑。

测试结果表明，系统能够在离线状态下完成资料保存、任务创建、番茄专注和复盘统计等核心操作；在后端服务可用时，可完成登录、数据拉取和数据上推。通过补充前端快速添加入口、同步连接提示、白噪音资源加载和构建脚本，系统的可用性和演示稳定性得到提升。客户端 HAP/APP 构建、后端 Maven 构建验证、接口联调和模拟器运行验证共同构成项目交付依据。

关键词： HarmonyOS；ArkTS；MVVM；本地优先；学习管理

Abstract

With the widespread use of mobile Internet, smart devices and generative artificial intelligence, college students increasingly organize learning activities on mobile devices. Course materials, experiment tasks, web links, classroom notes and review plans are often scattered across different tools, which increases the cost of finding materials, maintaining focus and reviewing learning outcomes. Existing to-do, note-taking and Pomodoro applications usually provide isolated functions, but they rarely connect material organization, task decomposition, focused execution and review analytics into a continuous workflow. This thesis therefore focuses on building a stable, explainable and extensible mobile learning loop that remains usable under weak-network or offline conditions.

To address this problem, this thesis designs and implements ZhiXu, a HarmonyOS study-management application. The system follows a local-first architecture, uses the on-device relational database RDB as the core data source, stores runtime configurations with Preferences, and organizes model, view and control logic under the MVVM pattern. The application provides Today dashboard, task management, resource library, Pomodoro focus, review statistics and cloud synchronization. Study resources can be stored locally and, when remote AI is available, distilled into summaries and key points; the distilled result can be converted into tasks; completed Pomodoro sessions are persisted locally and then reflected in review statistics. The backend is implemented with Spring Boot, MyBatis and MySQL/H2 to provide user, project, task, Pomodoro and synchronization APIs.

The test results show that the system can complete core operations such as resource storage, task creation, Pomodoro focus and review statistics in an offline state. When the backend service is available, it supports login, data pulling and data pushing. By improving quick task creation, synchronization hints, white-noise resource loading and build scripts, the system achieves better usability and demonstration stability. Client HAP/APP builds, backend Maven build checks, API integration checks and emulator-based runtime checks are used as delivery verification evidence.

Keywords: HarmonyOS; ArkTS; MVVM; local-first; study management

目 录

摘要	I
Abstract	II
目录	III
第一章 序论.....	1
第一节 研究背景和意义	1
第二节 国内外研究现状	1
第三节 本文主要工作.....	2
第四节 论文组织结构.....	3
第二章 工具选型	4
第一节 开发语言选型.....	4
第二节 开发工具选型.....	5
第三节 前端框架选择.....	5
第四节 后端框架与数据库选择	6
第三章 系统设计	7
第一节 总体设计	7
第二节 模型部分设计.....	7
第三节 视图部分设计.....	9
第四节 控制部分设计.....	11
第五节 网络与数据库设计	11
第六节 本章小结	12
第四章 系统实现	14
第一节 工程结构与运行环境.....	14
第二节 主要功能实现.....	15
第三节 运行效果	17
第四节 关键问题与解决方法.....	20
第五节 运行期性能与稳定性调优.....	20
第六节 测试与验证	22

第七节 本章小结	22
第五章 总结与展望	23
第一节 总结	23
第二节 展望	23
附 录 主要页面运行截图	24
第一节 运行截图	24
参考文献	33
致 谢	34

第一章 序论

第一节 研究背景和意义

移动互联网改变了大学生组织学习活动的方式。课程资料不再只来自教材，还包括课件、实验文档、网页链接、课堂笔记、课程群消息和复习清单等多种来源 [2, 11]。这些资料常分散在备忘录、网盘、浏览器收藏夹和聊天记录中，导致学生在真正开始学习时需要反复寻找上下文。与此同时，任务拆解、番茄专注和学习复盘往往被放在不同应用里完成，数据之间缺少稳定连接，学习过程难以形成连续闭环。

现有工具在局部场景上已经取得了较好效果。待办类工具适合记录任务和截止时间，笔记类工具适合保存资料，番茄钟工具适合维持注意力，AI 助手适合生成摘要和行动建议 [7, 8]。但是这些工具通常存在三个问题：第一，资料与任务分离，资料沉淀后不一定能转化为可执行行动；第二，专注过程与任务上下文分离，番茄记录难以反向支撑复盘；第三，远程 AI 和云端服务依赖网络环境，一旦处于弱网或离线状态，核心学习流程容易被中断。

HarmonyOS 提供了 ArkTS、ArkUI、RDB、Preferences、TaskPool、AVPlayer、ArkWeb 和 Ability 等端侧能力，为构建本地优先的移动学习应用提供了基础 [3–6]。本地优先思想强调核心数据和关键操作应优先保存在用户设备上，远程服务作为同步或增强能力存在 [9]。将该思想用于学习管理场景，可以使任务、资料、专注记录和复盘统计在离线状态下仍然可用，同时在网络可用时接入 AI 提炼和后端同步。

因此，本文设计并实现“知序”学习管理应用，目标是在 HarmonyOS 端侧建立“资料整理 - 任务分解 - 番茄执行 - 复盘反馈”的学习闭环。该研究具有实践意义：一方面，它能降低学生在多工具之间切换造成的认知成本；另一方面，它综合运用了移动端 UI、端侧数据库、原生能力调用、后端接口和数据库同步等课程知识，有助于验证移动应用开发课程的综合实践能力。

第二节 国内外研究现状

国内外关于学习管理和效率工具的研究与产品实践大体可以分为四类。第一类是待办和项目管理工具，例如 Microsoft To Do、滴答清单和 Trello 等，它们强调任务清单、截止日期和提醒机制，适合把目标拆解为行动项，但通常不直

接保存学习资料，也难以把资料阅读过程与任务执行记录关联起来。第二类是笔记和知识库工具，例如 Notion、OneNote 和语雀等，它们适合沉淀课程笔记、网页摘录和实验记录，但任务执行、番茄计时和复盘统计往往需要依赖外部工具完成。第三类是专注和番茄钟工具，例如 Forest、番茄 ToDo 等，它们通过时间块、视觉激励和中断控制帮助用户保持注意力，相关研究也表明番茄工作法对学习节奏管理具有积极作用 [1, 10]，但这类工具通常只记录时间，不理解时间背后的课程资料和任务上下文。第四类是 AI 学习助手和智能辅导系统，它们能够根据输入内容生成摘要、解释和建议 [7, 8]，但如果缺少本地数据组织和错误兜底，容易受网络、接口稳定性和隐私边界影响。

从研究角度看，移动学习已经被证明能够扩展学习发生的时间和空间 [2, 11]，但学习工具的碎片化仍然明显。学生常常需要在笔记、待办、计时器、网盘和聊天记录之间来回切换，导致资料、行动和结果分散。近年来 local-first 软件思想提出应优先保证用户在本地拥有数据和核心操作能力，云端更多承担同步与协作增强角色 [9]。这一思想与大学生学习场景高度契合：课程资料 and 任务应在离线状态下可访问，网络和 AI 服务则应作为增强能力而不是基本流程的前置条件。

综上，现有工具各有优势，但在“资料 - 任务 - 专注 - 复盘”一体化闭环方面仍有不足。本文的必要性在于：以 HarmonyOS 端侧数据库保证本地可用，以任务和番茄记录连接学习行动，以后端同步和 AI 提炼提供可选增强，从而在课程项目范围内实现一个功能完整、数据流清晰、可演示全栈能力的学习管理应用。

第三节 本文主要工作

本文围绕“知序”的设计与实现完成以下工作。

第一，分析大学生课程学习场景，明确资料、任务、专注和复盘四类核心对象，并确定系统必须满足离线可用、数据持久化、状态可解释和可扩展同步等要求。

第二，基于 HarmonyOS 与 ArkTS 构建移动端应用。客户端采用 MVVM 思路组织模型、视图和控制逻辑，使用 RDB 保存项目、任务、资料和番茄记录，使用 Preferences 保存计时器、登录态、AI 配置和同步配置。

第三，设计资料库与任务系统的闭环关系。用户可以保存笔记、链接、课件和文件引用；在 AI 服务可用时，系统可提炼资料摘要和要点，并进一步生成

任务；任务完成后写入番茄记录，最终形成复盘统计。

第四，实现 Spring Boot + MyBatis 后端，提供用户登录、项目管理、任务管理、番茄记录、统计和同步接口。后端既支持 MySQL 运行环境，也保留 H2 作为快速演示和测试数据库。

第五，完成前后端功能测试、模拟器观察、构建脚本修复和论文模板编译验证，并根据测试结果优化快速添加入口、同步提示、白噪音资源加载和启动说明。

第四节 论文组织结构

本文共五章。第一章为序论，介绍研究背景、国内外研究现状、主要工作和论文结构。第二章为工具选型，对开发语言、开发工具、前端框架、后端框架和数据库进行比较，并说明最终选择。第三章为系统设计，基于 MVVM 模式说明总体架构、模型层、视图层、控制层、网络部分和数据库设计。第四章为系统实现，说明主要功能的实现过程、运行效果、关键问题、性能调优和验证方法。第五章为总结与展望，总结本文工作并说明后续可改进方向。

第二章 工具选型

第一节 开发语言选型

移动端开发语言主要可选 Java/Kotlin、Dart、JavaScript/TypeScript 和 ArkTS。Java/Kotlin 在 Android 生态中成熟，但不能直接体现 HarmonyOS 原生开发特性；Dart 适合 Flutter 跨平台开发，界面一致性较好，但在调用 HarmonyOS 端侧能力时需要额外适配；JavaScript/TypeScript 具有较强表达能力，但普通 TypeScript 缺少 HarmonyOS 声明式 UI 和系统能力约束；ArkTS 在 TypeScript 基础上面向 HarmonyOS 做了增强，能够直接使用 ArkUI、Ability、RDB 和系统服务接口。

综合课程要求和平台适配，本文客户端选择 ArkTS。该选择的优点是能够充分调用 HarmonyOS 原生能力，界面与状态绑定方式清晰，适合实现本地优先移动应用；不足是生态资料相对 Android 和 Web 更少，构建、签名和模拟器调试对开发环境要求更高。

后端语言主要可选 Java、Python 和 Node.js。Python 开发速度快，但在企业级分层和静态类型约束方面不如 Java；Node.js 适合高并发 I/O 和前后端同语言开发，但课程中数据库持久化和典型三层结构展示不如 Java 直观。Java 与 Spring Boot、MyBatis 结合成熟，适合构建 RESTful 接口和数据库访问层。因此，本文后端选择 Java。

表 2.1 开发语言横向对比

语言	优点	不足	适用性
ArkTS	原生支持 HarmonyOS 声明式 UI、Ability、RDB 和系统服务	生态资料相对较少，构建环境要求较高	客户端首选
Java/Kotlin	Android 生态成熟，资料丰富	不能直接体现 HarmonyOS 原生能力	不采用
Dart	跨平台 UI 一致性较好	访问 HarmonyOS 端侧能力需要额外适配	不采用
Java	Spring Boot 和 MyBatis 生态成熟，适合 REST 接口和数据库访问	语法相对繁琐，需要 Maven 管理依赖	后端首选

第二节 开发工具选型

HarmonyOS 客户端开发工具主要可选 DevEco Studio 和通用代码编辑器。通用编辑器启动快、插件灵活，但缺少 HarmonyOS 模拟器、签名、HAP 打包和设备调试的一体化支持。DevEco Studio 集成 SDK、模拟器、Hvigor 构建、签名和 ArkTS 检查能力，是 HarmonyOS 应用开发的主要工具。因此，本文客户端开发选择 DevEco Studio。

后端开发工具可选 Eclipse、IntelliJ IDEA 和 Visual Studio Code。Eclipse 体积较轻，但 Spring Boot 项目的依赖管理和调试体验相对弱；Visual Studio Code 插件丰富，适合轻量编辑，但大型 Java 项目的重构和调试能力有限；IntelliJ IDEA 对 Maven、Spring Boot、MyBatis 和 Java 调试支持较完善。因此，后端开发更适合使用 IntelliJ IDEA 或 DevEco Studio 外部配合命令行 Maven。

在构建工具方面，前端采用 Hvigor，后端采用 Maven。Hvigor 是 HarmonyOS 工程默认构建工具，负责 ArkTS 编译、资源处理、HAP 打包和 APP 打包。Maven 负责后端依赖管理、测试和 JAR 打包，便于在不同机器上复现构建过程。

表 2.2 开发工具横向对比

工具	优点	不足	选择结果
DevEco Studio	集成 HarmonyOS SDK、模拟器、签名、Hvigor 和 ArkTS 检查	启动较重，对本机环境敏感	前端采用
通用编辑器	启动快，插件灵活	缺少完整 HarmonyOS 构建和设备调试链路	辅助编辑
IntelliJ IDEA	Java、Maven、Spring Boot 支持完善	需要额外安装和配置	后端推荐
Eclipse	轻量，课程后端示例常见	Spring Boot 和重构体验相对弱	备选

第三节 前端框架选择

HarmonyOS 前端可以选择 ArkUI 原生声明式开发，也可以通过 WebView 嵌入 H5 页面。H5 开发成本较低，图表和样式生态丰富，但端侧数据库、原生通知、文件选择、音频播放和 Ability 交互需要桥接，整体可靠性较低。ArkUI 能直接访问 HarmonyOS 端侧能力，状态驱动界面更新，适合构建与系统能力结合紧密的应用。

本文选择 ArkUI 作为主界面框架，并仅在复盘图表部分使用 ArkWeb 嵌入 Canvas 图表。这样可以兼顾原生交互和图表表达：任务、资料、专注和设置等

高频操作由 ArkUI 实现，减少桥接成本；统计图表利用 Web 技术绘制，降低图形渲染实现复杂度。

表 2.3 前端框架横向对比

方案	优点	不足	本文用法
ArkUI	原生能力调用直接，声明式状态更新清晰，适合移动端交互	复杂图表需要自行实现或引入 Web 能力	主界面采用
ArkWeb/H5	图表和 Canvas 生态成熟，页面表达灵活	与 RDB、Ability、音频等端侧能力交互需要桥接	仅用于复盘图表
纯 Web App	开发快，跨平台成本低	不能体现原生 HarmonyOS 能力	不采用

第四节 后端框架与数据库选择

后端框架主要比较 Spring Boot、Express 和 Flask。Express 与 Flask 轻量灵活，适合快速原型，但在分层约定、数据库映射和工程规范方面需要自行补充。Spring Boot 提供成熟的依赖管理、配置体系和 REST 接口开发方式，与 MyBatis 结合后可清晰展示 Controller、Mapper 和 XML SQL 的分层结构。因此，本文后端选择 Spring Boot + MyBatis。

数据库方面，移动端使用 HarmonyOS RDB。它是端侧关系型数据库，适合保存任务、资料、番茄记录等结构化数据。后端数据库提供 MySQL 和 H2 两种运行方式。MySQL 适合正式演示服务器和课程要求中的关系型数据库展示；H2 文件数据库适合快速测试和无需额外安装数据库的场景。为满足演示稳定性，系统保留两种配置：默认启动脚本支持 MySQL，测试时可通过参数切换到 H2。

表 2.4 后端框架与数据库横向对比

方案	优点	不足	选择结果
Spring Boot + MyBatis	分层清晰，REST 接口和 XML SQL 映射直观，适合课程全栈展示	初始配置多于轻量框架	后端采用
Express/Flask	轻量，原型开发快	数据库映射和工程规范需要自行约束	不采用
MySQL	适合正式演示和复杂关系型数据库展示	本机需要提前安装和初始化	默认 profile
H2	无需额外数据库服务，适合快速测试	与正式 MySQL 环境存在差异	测试 fallback

第三章 系统设计

第一节 总体设计

“知序”采用 MVVM 开发模式，将应用程序分为模型、视图、控制三大部分。模型部分负责描述业务对象和持久化结构，包括项目、任务、资料、番茄记录、设置和用户登录信息。视图部分负责页面展示和用户交互，包括今日、任务、资料、专注、复盘、我的、设置和同步等界面。控制部分负责连接视图与模型，处理用户操作、状态刷新、数据写入、AI 请求、后端同步和原生能力调用。

采用 MVVM 模式的优点在于：第一，页面不直接拼接数据库语句，降低界面和数据层耦合；第二，状态变化可以集中通过 Store 和服务层管理，减少不同页面之间的数据不一致；第三，后续增加后端同步、AI 提炼或新页面时，可以在服务层扩展能力，而不破坏已有视图结构。系统总体架构如图 3.1 所示。

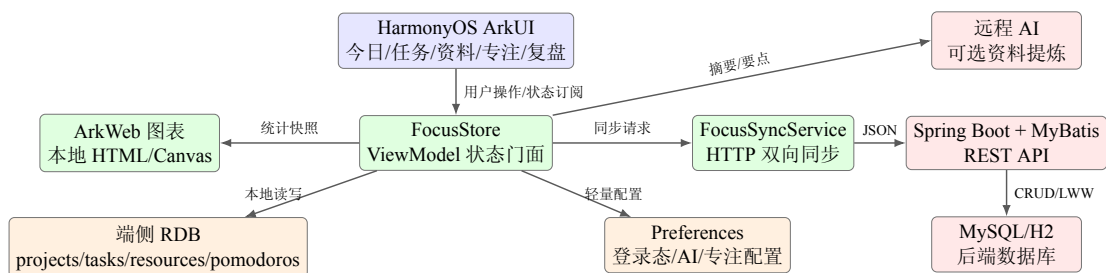


图 3.1 系统总体架构与数据流向

从学习流程角度看，系统不是把待办、资料和番茄钟简单并列，而是把它们组织为一个可循环的数据闭环。资料输入后可以被提炼为摘要和行动项，行动项进入任务池，任务执行产生番茄记录，番茄记录再进入复盘统计，复盘结果反过来影响后续任务安排。该流程如图 3.2 所示。

第二节 模型部分设计

模型部分围绕四类核心实体展开。项目用于组织课程或学习主题，包含名称、颜色和图标等信息。任务用于承载具体行动，包含标题、备注、所属项目、优先级、截止时间、状态、番茄数量、标签和子任务等信息。资料用于保存学习素材，包含标题、类型、正文摘录、摘要、要点、关联任务和复习时间。番茄记录用于描述一次专注过程，包含任务标识、开始时间、持续时长、是否完成和中断原因。

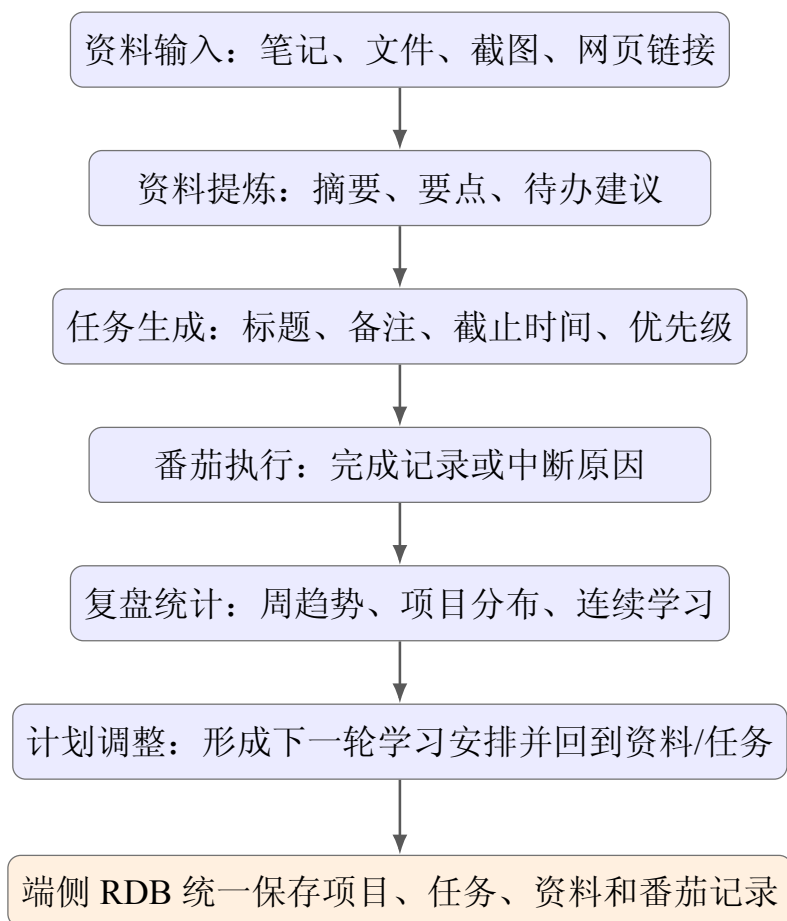


图 3.2 学习闭环业务流程设计

为了保证离线可用，模型数据首先写入端侧 RDB。任务状态采用待办、完成、删除三态设计，其中删除为软删除，不直接物理移除数据。资料状态分为活跃和归档，便于用户整理长期资料。标签、子任务和资料要点属于半结构化内容，在 ArkTS 中表现为数组，在数据库中以 JSON 字符串保存，以减少表结构复杂度。

核心实体之间的关系如图 3.3 所示。项目是一类组织维度，任务和资料都可归属于项目；任务执行后产生番茄记录，资料可以关联任务或被转换为任务；用户信息主要存在于后端，用于区分同步数据归属。

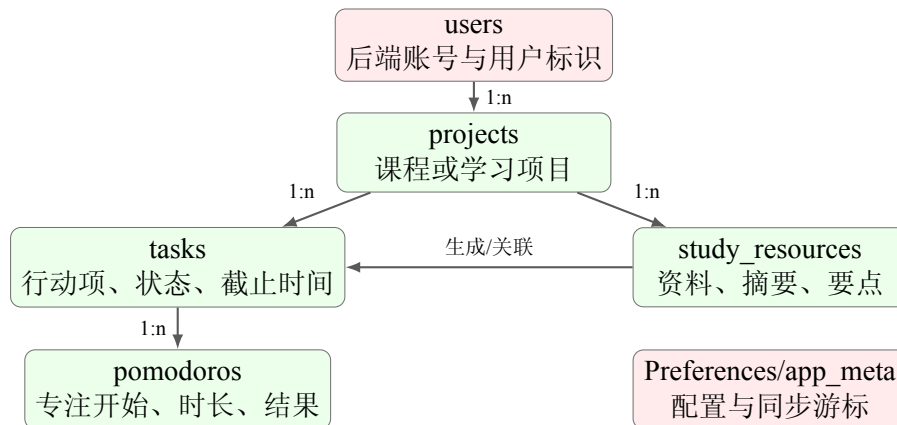


图 3.3 核心数据实体关系设计

设置类数据不适合放入业务表，因此使用 Preferences 保存。例如专注时长、白噪音类型、AI 接口地址、AI 模型、登录态、同步服务器地址等运行时配置分别存储在不同命名空间中。这样可以避免业务数据与配置数据相互污染，也便于后续清理或迁移。

第三节 视图部分设计

视图部分以底部标签页组织核心功能，并使用 HarmonyOS 推荐的 Navigation 组件管理二级页面。今日页作为学习工作台，展示当日进度、当前建议任务和待办列表；任务页负责快速添加、搜索、筛选、完成、删除和恢复任务；资料页负责资料录入、文件引用、资料筛选、AI 提炼和生成任务；专注页负责番茄计时、白噪音播放、中断记录和完成记录；复盘页展示本周专注时间、完成数量、项目分布和学习连续性；我的页整合复盘统计、资料库、云同步和设置入口。资料库、复盘统计、云同步和设置等入口通过 NavPathStack 压入对应 NavDestination，从而形成真实页面栈，而不是仅依赖条件渲染模拟页面切换。

为了降低页面文件的复杂度，视图层进一步把今日工作台拆分为 TodayView

组件，把任务页快速添加入口拆分为 QuickAddTaskCard 组件，把任务管理页拆分为 TasksView 组件，把任务列表项拆分为 TaskRowView 组件，把资料库工作区、资料录入、文件导入、AI 提炼和资料列表拆分为 ResourcesView 组件，把专注计时和偏好设置面板拆分为 FocusSessionContent 与 FocusSettingsPanel 组件，把“我的”页拆分为独立 ProfileView 组件，把复盘统计拆分为 ReviewView 组件，并把纯展示部件拆分为公共视觉组件，例如底部 Tab 图标、空状态、登录特性标签、通用图标徽章和学习森林植物图形等均由公共视觉组件负责。主页面只保留业务状态、用户操作入口和页面组合关系，公共视觉组件不直接访问数据库或网络服务。这样既符合 MVVM 中“视图负责展示、服务负责业务”的分工，也使资料、任务、专注和复盘等高频界面的代码边界更加清晰。

表 3.1 主要视图模块职责划分

模块	主要入口	职责边界
今日工作台	今日标签页	展示今日进度、推荐任务和近期待办，只负责聚合展示，不承担复杂录入。
任务管理组件	任务标签页	承担任务快速添加、图片提取、搜索筛选、完成切换、删除和恢复。
资料库组件	资料入口	组织资料保存、文件引用、AI 摘要、要点提炼和从资料生成任务。
专注计时组件	专注标签页/独立专注窗口	展示番茄计时、白噪音和中断记录，专注数据写入服务层。
复盘统计组件	复盘入口	读取统计快照并通过 ArkWeb/Canvas 展示趋势图和项目分布。
个人中心组件	我的标签页	聚合资料库、云同步、设置和复盘等高级入口。

界面设计遵循移动端高频操作优先原则。为避免今日页和任务页出现重复入口，快速添加任务集中保留在任务页；该入口支持多行输入、备注、截止时间、项目、优先级，以及拍照或上传图片后提取任务。考虑到图片识别容易受到截图清晰度、课程通知排版和模型响应稳定性的影响，图片入口采用“候选草稿确认后入库”的交互方案：优先使用端侧 OCR 获得文字，再调用远程 mimo-v2.5 模型抽取结构化任务；当端侧 OCR 不可用或未识别到文字时，再回退到图片 Base64 视觉请求。图片识别结果不会立即写入任务库，而是先展示为候选草稿，用户可以核对标题、备注和优先级后保存单条或全部候选任务。云同步页在未登录状态下明确提示模拟器与真机的服务器地址填写方式，避免用户把模拟器地址误用于真机。设置页将专注时长、白噪音和 AI 配置放在同一入口，保证影响专注体验的选项集中管理。

第四节 控制部分设计

控制部分以 FocusStore 和多个服务类为核心。FocusStore 作为页面状态门面，负责读取和刷新任务、项目、资料和番茄记录。页面在用户操作后调用 Store 或服务层方法，方法完成本地写入后刷新 Store，再同步到页面状态。该流程避免页面直接操作数据库，也减少多个页面状态不一致的问题。

设置服务负责读写 Preferences，并在用户修改专注时长或白噪音时立即保存。AI 服务负责保存模型配置、发起远程请求、解析响应并返回结构化建议。同步服务负责后端登录、拉取服务器数据、上推本地数据和更新同步时间。原生能力服务负责通知、后台保护和 AVPlayer 音频播放。

控制部分的关键原则是核心流程不依赖远程服务。任务新增、资料保存、番茄记录和复盘统计均在本地完成；AI 和后端只作为增强能力。如果远程 AI 未配置，系统只显示状态提示，不生成伪造结果；如果后端不可用，用户仍可继续使用本地学习闭环。

Ability 设计上，系统保留 EntryAbility 作为主入口，并将独立专注窗口拆分为 FocusAbility。FocusAbility 使用单例启动模式，避免用户多次进入专注时创建多个并行计时窗口；冷启动和热启动路径都围绕 Want 参数与页面加载进行处理，使从主界面、服务卡片或外部入口进入专注场景时能够落到一致的页面状态。EntryFormAbility 用于提供桌面服务卡片入口，卡片面向“今日进度”和“待办摘要”这类低频刷新信息，用户点击后可直接回到应用主流程。该设计对应 HarmonyOS 中 Application、AbilityStage、UIAbility、WindowStage 与 Page 的层级关系，也使主界面、专注窗口和桌面卡片的职责边界更加清晰。

第五节 网络与数据库设计

网络部分分为 AI 请求、后端同步和网络状态订阅三类。AI 请求兼容常见 Chat Completions 风格接口，客户端保存 endpoint、model 和 apiKey。请求失败、密钥缺失或响应格式异常时，服务层返回明确错误状态，视图层向用户展示提示。后端同步采用 RESTful 接口，主要包括用户登录、项目接口、任务接口、番茄接口、统计接口、同步拉取和同步上推。登录成功后，后端签发 HS256 JWT，移动端后续请求通过 Authorization 头携带令牌，后端拦截器校验后再将用户标识写入请求上下文，从而避免业务接口直接信任客户端传入的用户编号。后端同时接入 Swagger/OpenAPI，用于生成接口文档和课堂联调时的在线调试入口。云同步页通过 connection 模块监听网络可用、丢失和不可用事件，并在离线时灰

显登录和同步入口，避免用户在明显不可达的网络状态下反复触发失败请求。

后端数据库包含 `users`、`projects`、`tasks` 和 `pomodoros` 四类主要表。`users` 保存用户账号信息；`projects` 保存项目名称、颜色、图标、更新时间和删除标记；`tasks` 保存任务标题、备注、优先级、状态、截止时间、标签 JSON、子任务 JSON、更新时间、删除标记和客户端请求标识；`pomodoros` 保存番茄开始时间、持续时长、完成状态、中断原因、更新时间和客户端请求标识。删除标记用于在同步中传递 `tombstone`，客户端请求标识用于降低重复提交导致的重复记录风险。

表 3.2 客户端 RDB 表结构设计

表名	主要字段	存储内容
<code>projects</code>	<code>id</code> 、 <code>name</code> 、 <code>color</code> 、 <code>icon</code> 、 <code>updated_at</code> 、 <code>is_deleted</code> 、 <code>dirty</code>	保存课程或学习项目，用于组织任务和资料，并支持软删除、同步更新时间和本地待上推标记。
<code>tasks</code>	<code>id</code> 、 <code>title</code> 、 <code>project_id</code> 、 <code>status</code> 、 <code>po-</code> <code>modoro_count</code> 、 <code>updated_at</code> 、 <code>is_deleted</code> 、 <code>client_request_id</code> 、 <code>dirty</code> 、 <code>tags_json</code> 等	保存任务主体、优先级、截止时间、完成状态、番茄数量、删除墓碑、同步幂等标识、本地待上推标记、标签和子任务等信息。
<code>pomodoros</code>	<code>id</code> 、 <code>task_id</code> 、 <code>start_at</code> 、 <code>duration</code> 、 <code>completed</code> 、 <code>updated_at</code> 、 <code>client_request_id</code> 、 <code>dirty</code> 等	保存一次专注过程的开始时间、持续时间、完成状态、中断原因、同步幂等标识和本地待上推标记。
<code>study_resources</code>	<code>id</code> 、 <code>title</code> 、 <code>source_type</code> 、 <code>project_id</code> 、 <code>content</code> 、 <code>summary</code> 、 <code>key_points_json</code> 、 <code>status</code> 等	保存学习资料、AI 提炼结果、关联任务和复习时间。

客户端和后端数据库并不是简单替代关系。客户端 RDB 是移动端运行的可信数据源，后端数据库是同步和备份目标。这样设计可以保证应用在没有后端时仍能运行，同时在需要跨设备或课程全栈展示时具备服务器扩展能力。

第六节 本章小结

本章从总体架构、模型、视图、控制、网络和数据库角度说明了“知序”的系统设计。系统以 MVVM 模式降低界面、业务和数据之间的耦合，以本地优先思想保证离线可用，以后端同步和 AI 服务提供可选增强能力。

表 3.3 后端数据库表结构设计

表名	主要字段	存储内容
users	id、username、password_hash、nickname、created_at	保存用户账号、密码哈希、昵称和创建时间。
projects	id、user_id、name、updated_at、is_deleted 等	按用户保存项目数据，并通过更新时间和删除标记支持增量同步。
tasks	id、user_id、title、project_id、status、updated_at、is_deleted、client_request_id 等	保存服务端任务数据，客户端请求标识用于降低重复提交风险，删除标记用于向其他端同步删除结果。
pomodoros	id、user_id、task_id、start_at、duration、completed、updated_at、client_request_id 等	保存服务端番茄记录，用于统计、幂等写入和跨设备同步。

第四章 系统实现

第一节 工程结构与运行环境

系统由 HarmonyOS 前端和 Spring Boot 后端组成。前端工程位于 entry 目录，主要代码集中在 pages、components、models、services、repository 和 utils 等模块中。pages 负责页面编排和路由目的地，components 保存可复用的纯展示组件，models 保存业务模型，services 封装数据库、设置、AI、同步、音频和原生能力，repository 处理登录相关数据，utils 保存格式化和设计常量。

后端工程位于 focus-server 目录，采用 Controller、Entity、Mapper、DTO 和 XML SQL 的结构组织。Controller 负责接收 HTTP 请求，Mapper 负责数据库访问，DTO 负责请求和响应数据传输。后端资源目录中同时提供 H2 和 MySQL 配置，便于在演示和测试之间切换。

运行环境方面，前端使用 DevEco Studio、HarmonyOS SDK 和 Hvigor 构建；后端使用 JDK 21 和 Maven 构建。为解决不同机器 Java 环境不一致的问题，项目提供后端环境脚本和前端构建脚本：后端脚本会自动选择可用 JDK 并设置项目本地 Maven；前端构建脚本强制使用 DevEco Studio 自带 JBR，避免系统 Java 注册表异常影响 PackageApp。

为便于说明实现代码与功能之间的对应关系，表 4.1 汇总了系统主要工程模块。可以看出，页面层只负责组织交互，数据库、AI、同步和音频等能力都沉淀在服务层，后端则按接口、实体和 Mapper 分层组织。完整工程路径已在代码目录中保留，正文表格只列出关键模块角色。

表 4.1 功能模块与代码实现映射

功能模块	主要代码位置	实现要点
页面编排与路由	Index 页面	底部标签页、二级页面栈、状态刷新和业务回调。
任务管理	TasksView 组件	快速添加、图片提取、筛选、完成、删除和恢复。
数据持久化	FocusDatabase 服务	RDB 建表、数据读写、迁移列和 dirty 标记。
状态门面	FocusStore 服务	汇总项目、任务、资料和番茄记录，向页面提供统一入口。
AI 与图片提取	FocusAiCoach 服务	保存模型配置，解析远程 JSON、OCR 文本和兜底文本。
云同步	FocusSync 服务	登录、pull、push、数据合并和同步游标维护。
后端接口	focus-server 分层目录	Controller、Mapper、DTO 和 Entity 分层实现 REST API。

第二节 主要功能实现

今日页实现了学习状态概览和待办聚合，保留“先做这一件”、今日进度和待办列表，使首页更聚焦于判断下一步行动。快速添加任务集中放在任务页，由 QuickAddTaskCard 负责录入标题、备注、截止时间、项目和优先级；文本框支持多行输入，第一行作为任务标题，后续行可转为备注或子任务，因而也支持用户直接输入自定义事件。在远程 AI 配置可用时，对于纸质笔记、课堂板书或截图，用户可以拍照或上传图片。前端先调用 HarmonyOS 端侧 OCR 识别文字，并把 OCR 文本交给配置的 mimo-v2.5 模型解析；若 OCR 无结果，则读取图片为 Base64 后发起视觉模型请求。模型优先返回 tasks 数组，截图中存在多个作业或复习事项时可拆成多个候选草稿；用户确认后再通过 FocusStore 写入本地 RDB，从而降低图片识别误差对任务库的影响。

图片生成任务的实现流程如图 4.1 所示。该流程把“图片选择”“识别解析”和“写入任务库”拆成三个阶段，避免远程模型返回异常时直接污染任务数据；同时保留 OCR 文本兜底，使远程 AI 返回为空时仍能给出可检查的候选草稿。

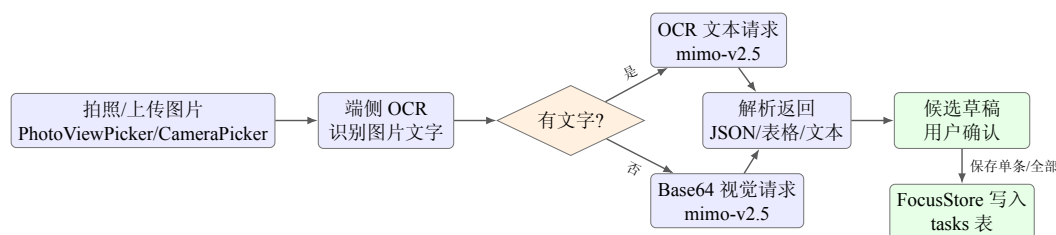


图 4.1 图片提取任务实现流程

任务管理实现了搜索、项目筛选、优先级筛选、完成切换、软删除和恢复。任务页的搜索、筛选和回收站状态由 TasksView 管理，单条任务的展示、完成切换、开始专注、删除和恢复入口由 TaskRowView 复用。任务完成时更新状态和完成时间；删除时进入回收站而不是直接移除；恢复时重新回到待办列表。这种三态设计既提升误操作容错，也为后端同步保留删除语义。

资料库实现了手动资料录入和文件引用保存。用户可以保存笔记、网页链接、课件说明、PDF 摘录或本地文件 URI。该模块的视图由 ResourcesView 统一承接，内部按照“资料库、备忘录、文件导入、AI 提炼”四个工作区组织界面，主页面只通过属性和回调传入状态及业务操作。系统只保存文件引用和用户备注，不直接复制大文件或解析文档全文，从而降低移动端权限和存储复杂度。资料被选中后，可以在 AI 配置可用时进行摘要和要点提炼，并将建议转换为任务。

专注模块实现了番茄计时、完成记录、中断记录和白噪音播放。用户开始专

注时，系统根据设置启动计时器和环境音；计时逻辑记录开始或恢复时的时间戳，并在界面刷新时根据当前时间计算剩余时长，定时器只承担刷新 UI 的职责，从而减少切后台或熄屏后单纯累减计时带来的漂移。完成或中断时系统停止音频并写入番茄记录。白噪音使用 rawfile 中的本地 MP3 文件，通过 AVPlayer 循环播放，避免依赖网络音频资源。

复盘模块根据本地任务和番茄记录计算统计数据，包括完成番茄数、专注分钟数、连续学习天数和周统计趋势。图表部分通过 ArkWeb 加载本地 HTML，并用 Canvas 绘制统计结果。云同步模块提供服务器地址、账号密码、登录和双向同步入口，登录成功后可执行先拉取后上推的数据同步流程。同步页同时订阅系统网络状态，网络断开时展示离线提示并暂停登录、同步按钮；网络恢复后再开放操作，从而减少弱网状态下的无效请求。

双向同步流程如图 4.2 所示。客户端不是每次上传全部数据，而是先用服务端时间游标拉取增量，再读取本地 dirty 数据上推；服务端根据 id、clientId 和 updatedAt 判断插入或更新，上推成功后客户端清除 dirty 标记并保存新的服务端时间。



图 4.2 云端双向同步实现流程

页面跳转实现采用 Navigation 与 NavPathStack。底部 Tabs 负责一级功能切换，资料详情、复盘统计、云同步和设置等二级入口通过 NavDestination 管理返

回栈。与简单布尔条件渲染相比，页面栈方式更接近 HarmonyOS 当前推荐的路由模型，用户从“我的”页进入二级页面后可以通过返回动作回到原入口，也便于答辩时说明路由结构和页面生命周期边界。

服务卡片通过 `EntryFormAbility` 提供桌面入口。主应用刷新数据后把今日待办、今日番茄进度和资料数量写入轻量 `Preferences` 快照；卡片创建或更新时读取该快照并绑定到卡片页面，避免在 `Form` 扩展中直接初始化完整数据库。这样既能在桌面展示真实业务数据，又降低卡片生命周期和数据库初始化之间的耦合。

第三节 运行效果

图 4.3 展示了今日页运行效果。该页面承担应用首页职责，集中展示当前学习状态、推荐执行任务和近期安排，使用户进入应用后可以直接判断当前任务状态并开始下一步行动。



图 4.3 今日页运行效果

图 4.4 展示了专注页运行效果。该页面围绕番茄计时、白噪音播放、中断记录和完成记录组织操作，计时状态与任务上下文绑定，能够把一次专注过程写入后续复盘统计。



图 4.4 专注页运行效果

图 4.5 展示了复盘页运行效果。该页面聚合本周专注时间、番茄完成数量、项目分布和连续学习情况，并通过 ArkWeb 承载本地图表展示，体现学习结果分析能力。

除上述页面外，任务页提供任务列表、筛选条件和任务操作入口，适合集中处理课程任务、实验任务和复习任务；我的页集中放置复盘统计、学习资料库、云同步和设置入口，适合作为高级功能与系统配置的统一入口。完整页面截图见附录 A。



图 4.5 复盘页运行效果

第四节 关键问题与解决方法

实现过程中遇到的第一个问题是前端最终 PackageApp 被系统 Java 注册表异常阻塞。ArkTS 编译和 PackageHap 可以通过，但 PackageApp 阶段调用到系统 Java 时会受到错误注册表配置影响。解决方法是增加专门的前端构建脚本，在脚本中设置 JAVA_HOME 和 PATH，强制使用 DevEco Studio 自带 JBR，并移除 Oracle javapath 的干扰。这样可以绕过系统 Java 注册表问题，使完整 APP 打包流程可复现。

第二个问题是后端环境脚本存在语法错误，导致测试启动前就被脚本拦截。解决方法是重写脚本，使其只负责设置项目自带 JDK 和 Maven，并在执行后输出 Java 与 Maven 版本。后端真正启动由 start-backend 脚本负责，从而区分“配置环境”和“启动服务”两个职责。

第三个问题是白噪音资源加载失败。原始音频资源使用中文文件名，在构建和资源访问过程中容易出现路径匹配问题。解决方法是将 rawfile 音频文件统一改为 ASCII 文件名，例如 outdoor.mp3、ocean.mp3 和 cricket.mp3，并在播放服务中维护枚举到文件名的映射。这样可以减少资源路径在不同工具链中的编码风险。

第四个问题是默认项目硬编码。早期快速添加和资料保存逻辑依赖固定项目 id，一旦种子数据或数据库状态变化，就可能写入错误项目。解决方法是改为按项目名称查找“移动开发”，找不到时使用第一个项目作为兜底，并让资料项目选择器的初始高亮与实际保存逻辑一致。

第五个问题是后端数据库选择带来的演示差异。MySQL 适合正式演示，但需要提前创建 focus_db 并导入初始化脚本；H2 启动成本低，适合自动化测试和课堂快速演示。项目保留两种配置，并在文档中说明默认 MySQL 与 H2 参数切换方式，以免测试阶段因本机未安装 MySQL 而误判后端不可用。

第五节 运行期性能与稳定性调优

运行期稳定性首先体现在异步调用边界处理。网络请求、AI 请求、后端同步、数据库初始化和资源加载都可能失败，因此系统将这些操作封装在服务层，并在调用过程中使用异常捕获和明确的状态返回。对于 HTTP 请求，服务层在请求完成或异常后释放请求对象，避免连接资源长时间占用。对于远程 AI，系统在密钥为空、网络失败或响应格式异常时只返回状态提示，不生成虚假的学习建议，从而保证核心学习流程不因外部服务波动而崩溃。图片提取任务进一步

提高输出 token 预算，并优先要求模型返回结构化 JSON；当模型返回 Markdown 表格、非标准字段或旧版六行格式时，服务层仍会使用文本解析兜底。若远程 AI 内容为空，界面会显示响应长度和重试状态；若请求超过 35 秒仍未返回，客户端会主动销毁 HTTP 请求并结束加载状态。对于已经识别出 OCR 文本的图片，即使远程 AI 返回为空，也会使用关键词和截止时间规则生成待确认草稿，便于定位是图片上传、模型输出还是解析环节出现问题。

本地数据库访问方面，系统使用 RDB 保存结构化数据，并在查询后释放 ResultSet。项目、任务、资料和番茄记录的读取过程均在服务层完成，页面只接收整理后的模型对象，避免界面层直接持有数据库游标。对于可能随版本变化的字段，系统使用表结构迁移思路按需补充列，降低旧数据升级时的失败风险。

并发与响应性方面，界面层通过 `async/await` 等异步机制调用数据库、网络 and 文件选择能力，使耗时操作不直接阻塞 UI 绘制。复盘页涉及周统计、项目分布、连续学习天数和植物解锁等多项聚合计算，系统在刷新数据后使用 `TaskPool` 对统计快照进行后台预计算，并保留同步计算回退路径；页面读取的是预计算后的结果，从而减少大量番茄记录下重复扫描数组对主线程的影响。云同步采用“先拉取、再上推”的串行流程，避免同一时间发起多个同步请求造成状态交叉；番茄记录和任务状态更新先在本地完成，再进入同步流程，使用户操作反馈优先于远程结果。同步游标以服务端返回的 `serverTime` 为基准，而不是直接使用客户端本地时间，从而降低两端时钟漂移造成漏拉数据的风险。本地项目、任务和番茄记录带有 `dirty` 标记，只有本地新增、修改或删除后的记录会参与上推，上推成功后再清除标记，减少数据量增长后每次同步全量上传带来的网络和后端写入压力。后端 `push` 接口使用事务包裹批量写入，避免项目、任务和番茄记录在中途异常时出现半写状态。对于 POST 非幂等问题，任务和番茄同步数据包包含稳定的 `clientRequestId`，可用于识别重复提交并降低重复写入风险。项目和任务删除并不直接丢弃同步信息，而是保留 `isDeleted` 删除墓碑，本地普通列表过滤墓碑，同步上推时仍会发送墓碑，使其他端不会在下一次拉取后重新恢复旧数据。客户端合并服务端项目、任务和番茄记录时会比较 `updatedAt`，只接收不旧于本地版本的数据；后端上推接口在发现相同 `id` 或 `clientRequestId` 时也会执行更新时间比较后再更新已有记录。该策略形成基于 `updatedAt` 的 Last-Write-Wins 裁决，避免多次同步产生重复任务、重复番茄或删除后“诈尸”的旧副本。

轻量配置数据使用 `Preferences` 保存，包括登录态、AI 配置、同步地址、专注时长和白噪音选项。`Preferences` 适合保存小规模键值数据，系统只在配置变

更后 flush，避免在高频 UI 更新中频繁落盘。白噪音播放使用本地 rawfile 文件，避免网络音频不稳定；播放服务在状态准备完成后再开始播放，并在停止时释放播放器和文件描述符，以减少资源泄漏。

第六节 测试与验证

后端接口联调覆盖项目、任务、番茄、统计和同步流程。使用 H2 模式启动并结合 Swagger/OpenAPI 或移动端同步页验证后，可以完成登录 demo/secret、项目列表查询、任务创建与更新、番茄记录创建、统计查询、同步拉取和同步上推。该结果说明后端接口和数据库映射可以支撑移动端同步功能。

前端模拟器观察覆盖今日页、任务页、我的页、任务完成切换、白噪音播放和图片上传入口。测试中任务完成状态能够影响今日进度，白噪音在专注时可以播放，说明本地状态刷新和 AVPlayer 调用有效。任务页验证显示，系统图片选择器可以正常打开并选中模拟器截图，返回 App 后能够先尝试端侧 OCR，再调用 mimo-v2.5 生成候选任务；若远程模型返回为空或超时，界面会展示明确诊断，并可在 OCR 文本有效时生成待确认草稿。用户确认后，候选草稿才会写入本地任务列表。对 UI 的进一步检查发现，快速添加入口集中放在任务页更符合功能边界，同步页应明确区分模拟器地址和真机局域网地址，这些问题已在实现中优化。

构建验证包括后端 Maven 构建检查、前端 HAP/APP 构建和论文 LaTeX 编译。后端脚本修复后可以自动选择可用 JDK 并执行 Maven 验证命令。前端构建脚本修复后可以通过 PackageApp。论文模板使用自动目录机制，目录由章节标题生成，而不是手工输入。

第七节 本章小结

本章说明了系统工程结构、核心功能实现、运行效果、关键问题和测试验证情况。实现结果表明，“知序”已经具备学习资料保存、任务管理、番茄专注、复盘统计和后端同步的基本闭环，并通过脚本和界面优化提升了项目演示稳定性。

第五章 总结与展望

第一节 总结

本文完成了基于 HarmonyOS 的本地优先学习管理应用“知序”的设计与实现。系统以 ArkTS 和 ArkUI 构建移动端界面，以 RDB 保存项目、任务、资料和番茄记录，以 Preferences 保存运行时配置，并按照 MVVM 模式组织模型、视图和控制逻辑。通过今日、任务、资料、专注、复盘和云同步等模块，系统将学习资料沉淀、任务执行、专注记录和复盘反馈连接为一个完整闭环。后端采用 Spring Boot 与 MyBatis 实现 RESTful 接口，并通过 MySQL/H2 支持正式演示和快速测试两种运行方式。测试结果表明，系统核心功能可以在离线状态下运行，后端可用时能够完成同步流程，整体满足课程项目对移动端开发、数据库设计、原生能力调用和全栈接口展示的要求。

第二节 展望

后续工作可以从五个方向继续完善。第一，增强资料处理能力，在文件引用基础上加入 PDF、DOCX、PPTX 正文解析和 OCR，使资料库能够处理更完整的学习材料。第二，提升同步能力，在当前 dirty 标记增量上推和 serverTime 游标的基础上，继续增加同步历史、失败重试、分页拉取和多设备状态提示，使后端同步更接近真实产品。第三，继续强化后端安全能力，在已有 JWT 登录校验基础上补充刷新令牌、权限角色和 HTTPS 部署，使认证机制更接近正式应用。第四，完善 AI 功能，加入结果缓存、提示词模板管理和更严格的结构化校验，降低外部服务波动对体验的影响。第五，继续优化前端体验，包括横屏和平板布局、无障碍适配、服务卡片交互动作、复盘图表交互和更细致的视觉层级，使应用从课程原型进一步接近可长期使用学习工具。

附录 主要页面运行截图

第一节 运行截图

本附录汇总系统主要页面的真实运行截图。截图用于说明系统已覆盖多个具有实际业务功能的页面，并展示资料、任务、专注、复盘和同步之间的数据闭环。所有截图均由模拟器或真机运行后截取，正文中已按图号引用。



今日页作为应用首页，集中展示当前学习状态、推荐执行任务和近期待办，便于用户打开应用后立即判断下一步行动。

图 A.1 今日页运行效果



任务页用于集中管理学习任务，支持搜索、项目与象限筛选、完成状态切换和回收恢复，体现任务拆解与执行闭环。

图 A.2 任务页运行效果



我的页整合复盘统计、资料库、云同步和系统设置入口，承担应用的统一配置与高级功能入口作用。

图 A.3 我的页运行效果



专注页展示番茄计时状态、剩余时长、白噪音与开始/结束控制，采用时间戳驱动的计时方式保证后台返回后仍然准确。

图 A.4 专注页运行效果



复盘页汇总周统计、项目分布、连续学习和专注记录，并通过 ArkWeb 展示图表区域，强化学习结果分析能力。

图 A.5 复盘页运行效果



资料页用于录入笔记、链接和本地文件引用，并在 AI 配置可用时进行提炼和任务转换，体现资料到行动的连接。

图 A.6 资料页运行效果



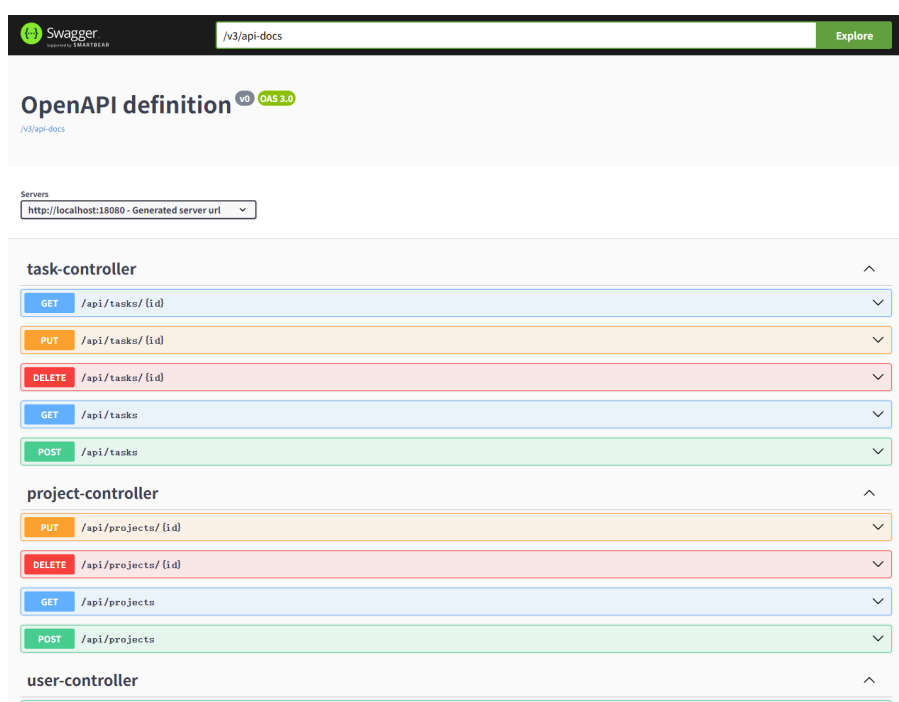
云同步页提供登录信息、服务器地址配置和同步操作入口，支持本地数据与后端服务之间的双向同步。

图 A.7 云同步页运行效果



桌面服务卡片展示今日待办、资料数量和番茄进度，用户可从系统桌面直接查看学习摘要并返回应用主流程，体现 EntryFormAbility 与轻量 Preferences 快照的结合。

图 A.8 桌面服务卡片运行效果



Swagger/OpenAPI 页面由后端服务运行后生成，用于展示项目、任务、番茄、统计、登录和同步等 RESTful 接口，便于课堂答辩和前后端联调时快速确认接口结构。

图 A.9 Swagger 接口文档运行效果

参考文献

- [1] Francesco Cirillo. The Pomodoro Technique. FC Garage, 2018.
- [2] Helen Crompton, Diane Burke. The use of mobile learning in higher education: A systematic review. *Computers & Education*, 2018, 123: 53–64.
- [3] HarmonyOS Developer. HarmonyOS Application Development Guide, 2024.
- [4] Huawei Developers. Loading Web Pages with ArkWeb Components, 2026.
- [5] Huawei Developers. Persisting Data with RDB Store, 2026.
- [6] Huawei Developers. TaskPool: Parallel Task Development, 2026.
- [7] Tumaini Kabudi, Ilias Pappas, Dag Håkon Olsen. AI-enabled adaptive learning systems: A systematic mapping of the literature. *Computers and Education: Artificial Intelligence*, 2021, 2: 100017.
- [8] Enkelejda Kasneci *et al.* ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education. *Learning and Individual Differences*, 2023, 103: 102274.
- [9] Martin Kleppmann *et al.* Local-first software: You own your data, in spite of the cloud. In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019.
- [10] Christian Santiago, Melanie Gurat. Effect of Pomodoro technique on student Mendelian genetics concept mastery during synchronous remote learning. *International Research Journal of Management, IT and Social Sciences*, 2023, 10(4).
- [11] Yi-Shian Sung, Kuo-En Chang, Tzu-Chien Liu. The effects of integrating mobile devices with teaching and learning on students' learning performance: A meta-analysis and research synthesis. *Computers & Education*, 2016, 94: 252–275.

致 谢

本课程设计报告的完成离不开许多人的帮助与支持。

首先，感谢我的指导教师在整个课程设计过程中给予的悉心指导和耐心帮助。从选题方向的确定、系统架构的设计到报告的撰写与修改，老师都提供了宝贵的意见和建议，使我在研究过程中少走了很多弯路。

其次，感谢移动应用开发课程的各位老师，课程中关于 ArkTS、ArkUI、HarmonyOS 端侧开发和后端接口设计的教学内容，为本项目的开发奠定了坚实的技术基础。

感谢南开大学计算机学院提供的学习环境和实验资源，使我能够在本科阶段完成这项综合性实践工作。

感谢我的同学们在项目开发过程中的交流与讨论，特别是在 HarmonyOS 开发实践中遇到的各种技术问题，大家的分享和帮助让我受益匪浅。

最后，感谢家人一直以来的理解与支持，是你们的鼓励让我能够专心完成学业和课程设计。